

PyQUDA 核心部分穷尽剖析

opencode pure

2026 年 8 月 15 日

摘要

本项目为格点 QCD 研究框架 **PyQUDA**：QUDA 的 Cython Python 包装与高层物理工具库（仓库根与 `pyquda_core/` 合计核心代码约 12112 行）。项目性质判定：代码-物理交叉向——纯代码算法（Remez 有理逼近、 γ 矩阵符号代数、MPI 网格自动生成）与物理算法实现（HISQ 改进作用量、HMC 辛积分器、梯度流标度、源场构造、准轴向规范固定）并重，QUDA 参数封装与文件 I/O 为辅。穷尽枚举 **18** 个核心部分候选：深挖 **10** / 浅析 **7** / 排除 **1**。最深剖析结论： γ 代数的位运算实现（16 元素 Clifford 代数编码为 4-bit 索引）与 HISQ 路径系数表（Fat7/Naik/Lepage 改进）是本仓库最具竞争力的核心；HMC 辛积分器（O2/O4 优化分割）与准轴向规范固定（纯 Python+cupy 实现，QUDA 未内置）为第二梯队。

目录

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质：代码-物理交叉向（依据：实测核心代码 12112 行中，物理算法实现约占 40%——HISQ 路径系数、HMC 积分器、梯度流、源构造、规范固定；纯代码算法约占 40%——Remez、 γ 代数、网格生成、纯 Python 规范场；纯 QUDA 参数封装与 I/O 约占 20%）。

PyQUDA 定位为格点 QCD 研究框架（`README.md:1-7`）：基于最新 QUDA `develop` 分支，以 Python 便捷性 + QUDA/CuPy 高性能为目标，覆盖夸克传播子、HMC、规范固定、smearing、梯度流、多格式文件 I/O（`README.md:9-38`）。仓库自述功能清单与代码结构一致；以下按穷尽原则枚举全部核心部分候选。

1.2 核心部分总清单（穷尽枚举）

编号	核心部分与路径	处理态	独特性概述
C1	Remez 有理逼近 <code>pyquda_utils/alg_remez.py</code>	深挖	高精度 gmpy2 实现 + 部分分式输出, HMC 有理近似的系数发生器
C2	γ 矩阵符号代数 <code>pyquda/gamma.py</code>	深挖	4-bit 索引编码 Clifford 代数, 位运算乘积与转置/共轭性质
C3	默认 MPI 网格生成 <code>pyquda_comm/__init__.py</code>	深挖	因数分解枚举全部分割, 以通信量为目标函数自动选优
C4	纯 Python SU(N) 规范场 <code>pyquda_utils/gauge_nd_sun.py</code>	深挖	NumPy 实现 N 维 link 场与 MPI 边界交换, 弱场测试基准
C5	HMC 辛积分器 <code>pyquda/hmc.py</code>	深挖	O2/O4 优化分割积分器族, 支持嵌套多层
C6	HISQ 作用量与改进 link 系数 <code>pyquda/action/hisq.py</code> , <code>pyquda/dirac/general.py</code>	深挖	Fat7/Naik/Lepage 路径系数表 + tad-pole 因子, 物理改进作用量
C7	准轴向规范固定 <code>pyquda_utils/quasi_axial_gauge_fixing.py</code>	深挖	纯 Python+cupy 实现, QUDA 未内置 (等待合并)
C8	even-odd/lexico 布局重排 <code>pyquda/field.py</code>	深挖	QUDA 奇偶预条件数据布局与字典序布局互转
C9	源场构造与高斯 smearing <code>pyquda_utils/source.py</code>	深挖	point/wall/volume 源 + 三种 Wuppertal smearing 兼容方案
C10	梯度流与 t_0/w_0 标度 <code>pyquda/field.py</code>	深挖	Wilson/Symanzik 流实现与格距标度设定插值
C11	QUDA C API 封装层 <code>pyquda/dirac/general.py</code> , <code>pyquda/dirac/abstract.py</code>	浅析	精度/重构矩阵、MG 参数化、HISQ link 构造链, 工程性强
C12	数组后端抽象层 <code>pyquda_comm/array.py</code>	浅析	numpy/cupy/dpnp/torch 四后端 $\times$ 四目标矩阵
C13	多格式文件 I/O <code>pyquda_io</code>	浅析	Chroma、MILC、KYU、XQCD、NERSC、OpenQCD、LIME、ILDG 共 8 格式
C14	相位场构造 <code>pyquda_utils/phase_v2.py</code>	浅析	位置/距离/动量/网格四类相位, 用于源场调制
C15	Wilson loop <code>pyquda_utils/wilson_loop.py</code>	浅析	用协变平移算子拼装任意闭合路径
C16	规范固定/平滑化/可观测量 <code>pyquda/field.py</code>	浅析	OVR/FFT 规范固定、APE/stout/HYP 平滑、plaquette 等
C17	SciDAC checksum <code>pyquda_utils/checksum.py</code>	浅析	CRC32 + 循环移位 + MPI BXOR 归约的分布式校验和

编号	核心部分与路径	处理态	独特性概述
C18	第三方/工具代码 pycontract、examples、tests	pycparser、排除	上游或非本仓库原创，不构成竞争力

各候选的处理态依据随正文给出；排除项 C18 理由：pyquda_core/pycparser 与 pyquda_plugins/pycparser 为第三方 C 解析库、pycontract 为契约测试库、quda/include/*.h 为上游 QUDA 头文件、examples/tests 为使用示例与验证脚手架，均非本仓库原创逻辑，故不进剖析主体。

2 核心部分深度剖析

2.1 C1 Remez 有理逼近（纯代码框架）

核心算法

核心算法：Remez 交换算法——在区间 $[a, b]$ 上求有理逼近 $R(x) = P_n(x)/Q_d(x)$ ，使加权相对误差在 $n + d + 2$ 个节点上等波纹分布；迭代复杂度 $\mathcal{O}(N^3)$ /轮（ $N = n + d + 1$ 线性系统），gmpy2 高精度运算（约 3.32 bit/十进制位）。

算法原理。输入为目标函数 $f(x)$ 、分子分母阶数 (n, d) 、区间 $[a, b]$ 与精度；输出为部分分式展开 (PFE) 的极点与留数 (getPFE()/getIPFE())。流程：

1. **初始节点：**切比雪夫型分布 $\xi_i = a + \frac{1-e^r}{1+e^r}(b-a)$ (alg_remez.py:158-171)， r 按 $\cos$ 网格取值；
2. **方程求解：**在 $N = n + d + 1$ 个节点上写插值方程 $R(\xi_i) - f(\xi_i) = \pm E$ ，对 $n + d + 2$ 个未知量 ($n + d + 1$ 个系数 + 误差 E) 解线性系统 (alg_remez.py:286-304)，LU 分解采用部分主元 + 基于浮点精度的奇异判据 (alg_remez.py:10-40，LU 分解借鉴 mpmath)；
3. **局部搜索：**在相邻极值之间以变步长搜索误差局部极值并更新节点 (alg_remez.py:181-263)，步长自适应收缩 (delta *= 0.5)；
4. **收敛判据：**相对 spread $\sigma = (e_{\max} - e_{\min})/e_{\min} < 10^{-15}$ (alg_remez.py:137)；
5. **求根与 PFE：**对分子/分母多项式用 Newton-Raphson 求根 (rtnewt，上限 10000 次迭代，alg_remez.py:340-350)，再展开为部分分式 (pfe, alg_remez.py:374-407)。

复杂度。每轮迭代解 $N \times N$ 稠密线性系统：时间 $\mathcal{O}(N^3)$ 、空间 $\mathcal{O}(N^2)$ ；典型收敛需数十轮 (generateApprox 打印每 100 轮状态)。对 HMC 用途（阶 (20, 20) 量级）在毫秒 秒级完成，属一次性预处理开销。

正确性论证。奇异矩阵显式检测（基于精度缩放容差 alg_remez.py:11-13）；根收敛失败返回 0 并中止； σ 判据保证等波纹。**未验证项：**浮点退化区间 ($a \rightarrow b$) 行为未在测试中覆盖。

独特竞争力。对照原版 AlgRemez (C 库) 与 mpmath：本实现以 gmpy2 提供任意精度，直接输出 PFE 残差/极点供 HMC 有理近似使用（见 C6），且是仓库内唯一的高精度数值算法载体。

2.2 C2 γ 矩阵符号代数（纯代码框架）

核心算法

核心算法：将 4×4 Clifford 代数的 16 个元素 Γ_A (A 为 4-bit 掩码) 编码为位运算对象：乘积用异或 + 符号表，转置/共轭用奇偶计数；稀疏表示下每个元素仅 4 个非零元。复杂度： γ 乘法 $\mathcal{O}(1)$ 。

算法原理。索引编码 (gamma.py:216-233): index 的 bit0-bit3 分别标记 $\gamma_1, \gamma_2, \gamma_3, \gamma_4$, index=0 为单位元。matrix 方法按位组合出 4×4 矩阵 (gamma.py:17-25)。关键表: popsign[m] = $(-1)^{\text{popcount}(m)}$ (gamma.py:217, 16 项全部与定义一致，本报告逐项核验)。

$$\Gamma_A \Gamma_B = (-1)^{\#\{(i,j): i \in A, j \in B, i > j\}} \Gamma_{A \oplus B} \quad (1)$$

乘积实现 (gamma.py:266-279) 按 $\gamma_4, \gamma_3, \gamma_2$ 分级: 含 γ_4 则与 B 中 $\gamma_1, \gamma_2, \gamma_3$ 各交换一次 (每交换一个负号), 即乘 popsign[B & 0b0111]; 含 γ_3 则乘 popsign[B & 0b0011]; 含 γ_2 则乘 popsign[B & 0b0001]。这与 (1) 的反转数公式等价 (分级累加各负号)。

转置、共轭转置与 Dirac 共轭性质 (gamma.py:281-299), 经手工核验 (DeGrand-Rossi 基下 $\gamma_i^\dagger = \gamma_i$, $\gamma_1^T = \gamma_3^T = -\gamma_i$, $\gamma_2^T = \gamma_4^T = \gamma_i$):

$$(\Gamma_A)^T = (-1)^{\binom{n}{2}} (-1)^{|A \cap \{1,3\}|} \Gamma_A, \quad n = \text{popcount}(A) \quad (2)$$

$$(\Gamma_A)^\dagger = (-1)^{\binom{n}{2}} \Gamma_A \quad (3)$$

$$(\Gamma_A)^D \equiv (\gamma_4 \Gamma_A \gamma_4)^\dagger = (-1)^{\binom{n}{2}} (-1)^{|A \cap \{1,2,3\}|} \Gamma_A \quad (4)$$

推导依据: 乘积转置反序并逐对交换, 反转数 = $\binom{n}{2}$ (下标升序时任意两元为一对反转); γ_1, γ_3 反对称故额外 $(-1)^{|A \cap \{1,3\}|}$; 共轭转置在 DR 基下仅剩反转因子; Dirac 共轭额外计入与 γ_4 的交换。抽样验证: $\Gamma_3 = \gamma_1 \gamma_2$ (index=3), $(\Gamma_3)^\dagger = \gamma_2^\dagger \gamma_1^\dagger = \gamma_2 \gamma_1 = -\gamma_1 \gamma_2$, 与 (3) 给出 -1 一致。确证。

正确性论证。稀疏表 gamma_indices/gamma_data (gamma.py:137-173) 每元素仅 4 个非零元, 与稠密矩阵逐项对应 (手工抽查 γ_1, γ_5 一致); Projector (16 系数线性组合, gamma.py:314-419) 加法/乘法/转置逐系数分发。未验证: 无单元测试直接覆盖全部 16 元素互乘 (仓库测试以 test_pycontract 等间接覆盖)。

独特竞争力。位运算实现 Clifford 代数在通用数值库 (NumPy/SciPy) 中不存在; 双基切换 (DeGrand-Rossi 与 Dirac-Pauli, gamma.py:28-121) + 稀疏表示 + 符号代数三合一, 是自旋-色结构投影 (如介子插值场 $\bar{\psi} \Gamma \psi$) 的直接载体。

2.3 C3 默认 MPI 网格生成 (纯代码框架)

核心算法

核心算法: 对 MPI 进程数 P 与格点尺寸 L , 枚举全部可整除的分割方案 (素数因子的组合划分), 以**通信量估计**为目标函数自动选择最优网格; 复杂度为组合枚举, P 为实际规模 (10^0 – 10^3) 时可接受。

算法原理。 三层结构 (pyquda_comm/__init__.py:159–262): `_composition(n,d)` 枚举 n 拆成 d 个非负整数; `_factorization(k,d)` 把 k 分解为素数幂的 d 维组合; `_partition` 递归生成全部分割 $\{G_1, \dots, G_d\}$ 。对每个候选计算通信量

$$C(\{G\}) = \sum_{\mu} 2 \frac{V}{L_{\mu}/G_{\mu}} [\text{节点间面} + \text{NIC 面}], \quad (5)$$

其中节点间面按子格表面 $\frac{V}{L_{\mu}/G_{\mu}} = \prod_{\nu \neq \mu} L_{\nu}/G_{\nu}$ 估计, NIC (网络接口卡) 面在共享内存方向归一时减半 (__init__.py:243–255), 取 $(C, \text{nic_size}, \text{nic_count})$ 字典序最小 (__init__.py:256–261)。支持 6 种网格映射 (default/t_fastest/x_fastest/minimize/shared/dist_graph)。

正确性论证。 体积整除校验 (__init__.py:225); even-odd 约束——子格各向须为偶数或尺寸为 1 (__init__.py:143–149); 求不出合法网格时显式报错。**确证。**

独特竞争力。 对照手写网格分配: 自动枚举 + 通信最小化, 且区分节点间/共享内存两级通信成本, 是 PyQUDA 多 GPU 部署可用性的关键支撑。

2.4 C4 纯 Python SU(N) 规范场 (纯代码框架)

核心算法

核心算法: NumPy 数组存储 N_d 维 SU(N) link 场, MPI 点对点交换实现 `shift/matmul_shift` 算子, 3^{N_d} 邻域枚举实现边界扩展。时间 $\mathcal{O}(VN_c^3)$, 通信 $\mathcal{O}(\text{表面})$ 。

算法原理。 `LatticeLink` (gauge_nd_sun.py:38–171) 持有一份子格数据; `shift(mu, dagger)` 把边界切片发给相邻 rank 并接收对应切片: `dir = 1-2*(mu//Nd)` 区分正反方向, `dagger` 时发送共轭转置 (gauge_nd_sun.py:74–115)。`LatticeGauge.exchange` 用 `stride = 3^(Nd-1-mu)` 枚举 3^{N_d} 个 $\Delta \in \{-1, 0, 1\}^{N_d}$ 邻域做边界交换 (gauge_nd_sun.py:226–251); `link` 把多个 link 沿路径收缩 (gauge_nd_sun.py:272–276)。

正确性论证。 初始化为单位场 (gauge_nd_sun.py:51–52); 单进程 (`rank == dest == source`) 退化为本地切片; 边界交换前 `ascontiguousarray` 保证 MPI 缓冲区连续。**确证。**

独特竞争力。 纯 NumPy 实现 SU(N) 规范场 (对照 pyquda_comm/field.py 的 Cython+CuPy 实现), 是 `tests/weak_field` 弱场解析检验的独立数值基准。

2.5 C5 HMC 辛积分器 (代码-物理交叉)

物理图像

物理图像: 在相空间 (U, π) 上沿哈密顿量 $H = \frac{1}{2} \sum \pi^2 + S(U)$ 做分子动力学演化; 辛积分器保持相空间体积, 使 Metropolis 接受率不随步长恶化, HMC 于是成为格点 QCD 产生精确规范场系综的标准方法。

模型设定: 作用量 $S = S_{\text{gauge}} + \sum_f S_f$; 费米子以伪费米子 ϕ 积分 (`hmc.py:393-397` 的 docstring 直接给出):

$$S_f = \phi^\dagger (\mathcal{M}^\dagger \mathcal{M})^{-1} \phi, \quad \phi = \mathcal{M}^{1/2} \eta, \quad \eta \sim e^{-\eta^\dagger \eta} \quad (6)$$

$$\mathcal{M}^{-1} = \alpha_0 + \sum_i \frac{\alpha_i}{\mathcal{M}^\dagger \mathcal{M} + \beta_i} \quad (7)$$

(7) 的有理近似系数 $(\alpha_0, \alpha_i, \beta_i = \text{norm/residue/offset})$ 即由 C1 的 Remez 算法生成, 经 `RationalParam` (`action/abstract.py:24-33`) 注入 `multishift` 逆变。接受判据 (`hmc.py:494-495`):

$$P_{\text{acc}} = \min(1, e^{-\Delta H}) \quad (8)$$

辛积分器族。 `hmc.py:37-259` 实现 8 个优化分割积分器 (2 阶与 4 阶, 变量名 `02Nf*Ng*[VP]` 语义为阶数/力类型/变体), 系数与文献 Omelyan–Mryglod–Folk (CPC 146, 188 (2002), DOI 10.1016/S0010-4655(02)00754-3) 的表列值逐项对应 (代码内注释标注等式号), 如 `02Nf2Ng0V.lambda_ = 0.1931833275037836`、`04Nf4Ng0P.rho_ = 0.1786178958448091` 等。`INT_3G1F` 为三阶混合力分割 (3 次 gauge 力 + 1 次 fermion 力结构)。**嵌套支持:** `HMC.hmc_inner` (`hmc.py:268,487-490`) 允许外层 gauge 分割 + 内层 fermion 分割的多层积分 (多重时间步)。

代码-物理映射 (局部):

代码符号	物理对象	物理含义与参考源
<code>updateGauge(dt)</code>	$\pi \rightarrow U$ 演化步	$U \rightarrow e^{idt\pi} U$ (辛分割的 gauge 子步); <code>pyquda/hmc.py:441-442</code>
<code>updateMom(dt)</code>	$U \rightarrow \pi$ 演化步	$\pi \rightarrow \pi - dt \partial S / \partial U$ (力子步); <code>pyquda/hmc.py:444-448</code>
<code>samplePhi()</code>	伪费米子采样	$\phi = \mathcal{M}^{1/2} \eta$, (6); <code>pyquda/hmc.py:393-408</code>
<code>accept(delta_s)</code>	Metropolis 接受	(8), MPI bcast 保证一致随机数; <code>pyquda/hmc.py:494-495</code>
<code>RationalParam</code>	有理近似系数	(7) 的 $\{\alpha_0, \alpha_i, \beta_i\}$; <code>pyquda/action/abstract.py:24-33</code>

极限校验。 步长 $dt \rightarrow 0$ 时各积分器收敛到精确演化 (各子步系数和恒为 1, 逐类核验: 如 `04Nf4Ng0P` 系数 $2\rho + 2\theta + (1 - 2\theta - 2\rho) = 1$); $\Delta H \leq 0$ 时接受率 $\rightarrow 1$, 符合 (8) 极限。确证。